SECURITY

Password Hashing Visual Guide

Store passwords so a stolen database isn't a stolen password list.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Why Not Plaintext (or Plain SHA)

Databases leak. Hashes must survive the leak.

Plaintext

breach = every password

Unsalted SHA-256

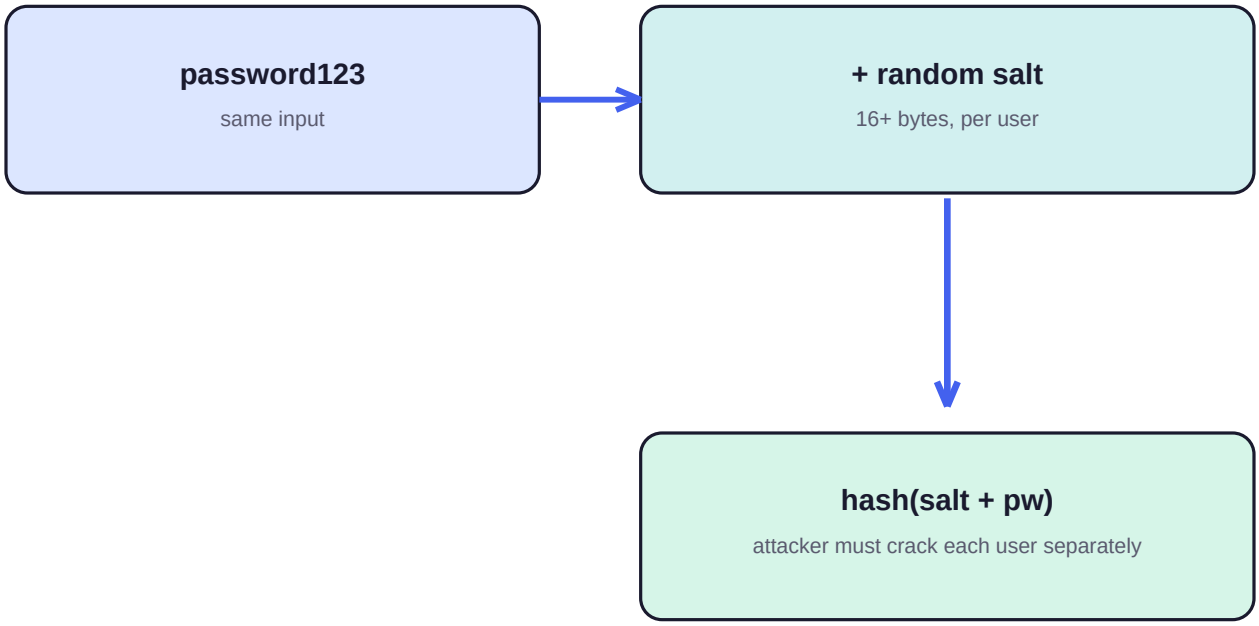
rainbow tables crack it in seconds

Fast hashes are the enemy: GPUs compute billions of SHA-256 guesses per second. Password hashing must be deliberately SLOW.

STEP 2/6

Salting: Unique Hash per User

Same password, different stored hash.



STEP 3/6

The Right Algorithms

Memory-hard + tunable cost. Pick one.

Argon2id
modern default — memory-hard, tunable

bcrypt
battle-tested, cost factor

scrypt
memory-hard, older tuning

Never MD5 / SHA-1 / SHA-256 alone for passwords — they're built for speed, which is exactly what attackers want.

STEP 4/6

Work Factors: Tune the Cost

Slow enough to hurt attackers, fast enough for users.

Target: ~0.5–1s

per login on your hardware

Attacker cost

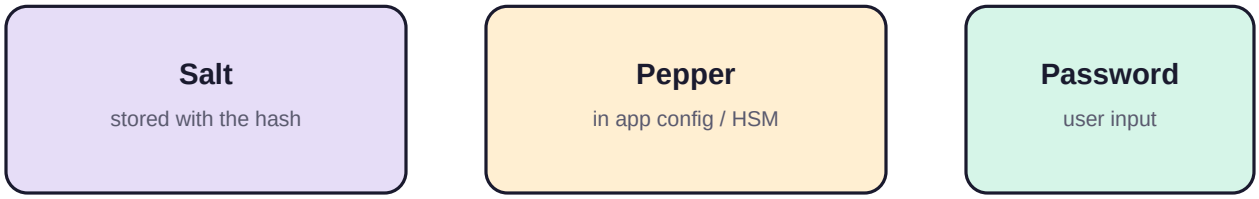
× millions of guesses = years

- Argon2id: tune memory (e.g. 64MB), iterations, parallelism together.
- bcrypt: cost 12+ as a starting point; raise as hardware improves.
- Re-tune annually — what was expensive in 2020 is cheap on 2026 GPUs.

STEP 5/6

Pepper: The Extra Layer

A secret salt the database never sees.



Pepper lives outside the DB (config, env, HSM/KMS) — a pure database dump is then useless without it. Rotate carefully: you must re-hash.

STEP 6/6

Migrating Old Hashes

Upgrade without forcing a mass password reset.

- On each login: verify against the old hash, then immediately re-hash with Argon2id.
- Flag legacy-hash users; expire stragglers with a forced reset after a window.
- Never decrypt-then-rehash — you can't decrypt a hash; the login moment is your only chance.
- Monitor: track the % of users still on legacy hashes until it hits zero.

STEP ✓I✓

Key Takeaways

Password Hashing — Remember

- Never store plaintext; never use fast hashes (MD5/SHA) alone.
- Salt every password with a unique random salt — kills rainbow tables.
- Use Argon2id (or bcrypt/scrypt) with a tuned work factor (~0.5–1s).
- Add a pepper stored outside the database for dump-resistance.
- Migrate legacy hashes opportunistically at login; re-tune cost yearly.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev